Simulador de movimiento de multitudes basado en agentes

Gabriel Astudillo Muñoz, gabriel.astudillo.m@usach.cl

17 de enero de 2019

1. Introducción

La Simulación Basada en Agentes (ABS, Agent-Based Simulation) o o Modelado Basado en Agentes (ABM, Agent-Based Model), es una técnica que permite modelar sistemas complejos que están compuestos por agentes interactivos y autónomos, desde una metodología Bottom-Up (Quezada & Canesa, 2010). Esta herramienta ha sido utilizada además para investigar y entender la dinámica de los fenómenos geoespaciales (Fachada et al., 2017; Li et al., 2016). Un ABM, normalmente consta de tres elementos: Agentes, Relaciones entre ellos y el Ambiente donde se ubican. Los comportamientos de estos agentes se describen mediante reglas e interacciones con otros agentes en el sistema. Estas reglas e interacciones a su vez influyen en sus comportamientos. Al modelar los agentes individualmente, tanto en sus atributos como en su comportamiento, se pueden observar los efectos que producen sus interacciones en el sistema completo, como patrones, estructuras y comportamientos que no estaban explícitamente programados (Macal & North, 2015). En el caso de los ABM geoespaciales enfocados a las evacuaciones masivas, estos comportamientos emergentes pueden ser utilizados para identificar problemas en el proceso de evacuación, mejorar procedimientos de evacuación y reducir los tiempos en que las personas tardan en llegar a los lugares de encuentro.

El tamaño de los problemas estudiados con ABM sigue incrementándose, al igual que la complejidad de ellos (Li et al., 2016). Esto hace que dichos modelos sean demandantes desde el punto de vista computacional. Para que sean útiles, se requiere que se ejecuten lo suficientemente rápido como para permitir un análisis comprensivo y sistemático (Thiele et al., 2014).

Dada esta necesidad de ejecutar varias simulaciones en un tiempo razonable, el problema entonces es cómo mantener el tiempo de ejecución manejable mientras se escala a un mayor número de agentes. El tiempo de ejecución de una simulación del modelo, depende de su diseño: cómo son las características de los agentes, la complejidad computacional de su comportamiento y la cantidad de agentes. Un ABM serial típico recorre todos los agentes en cada paso de tiempo y hace que cada agente ejecute algún comportamiento. El tiempo para ejecutar este ciclo depende de la cantidad de agentes que se deben recorrer y la complejidad del comportamiento de cada uno de ellos. En consecuencia, a medida que un ABM se amplía a poblaciones de tamaño y complejidad crecientes, es más susceptible a aumentos prohibitivos en el tiempo de ejecución como resultado del tiempo que lleva completar este ciclo de comportamiento. Además,

también es susceptible a problemas de memoria, lo que en última instancia limita el número de agentes posibles en un modelo.

En este trabajo se presenta una nueva versión del software DEMPS (Disaster Evacuation and Mobility Patterns Simulator). El propósito de este software es simular el comportamiento de personas en situaciones de evacuación masiva, bajo el paradigma de ABM, siguiendo las pautas de implementación de Macal & North (2015) para este tipo de software. Utiliza el motor de enrutamiento OSRM (Luxen & Vetter, 2011) para los desplazamientos de los agentes en un mapa determinado y el Modelo de Fuerza Social, propuesto por Helbing & Molnar (1998), que establece que los cambios de comportamiento en el movimiento humano pueden explicarse por la combinación de sus interacciones sociales, objetivos individuales y la estructura física donde se desenvuelven.

2. Descripción global del software

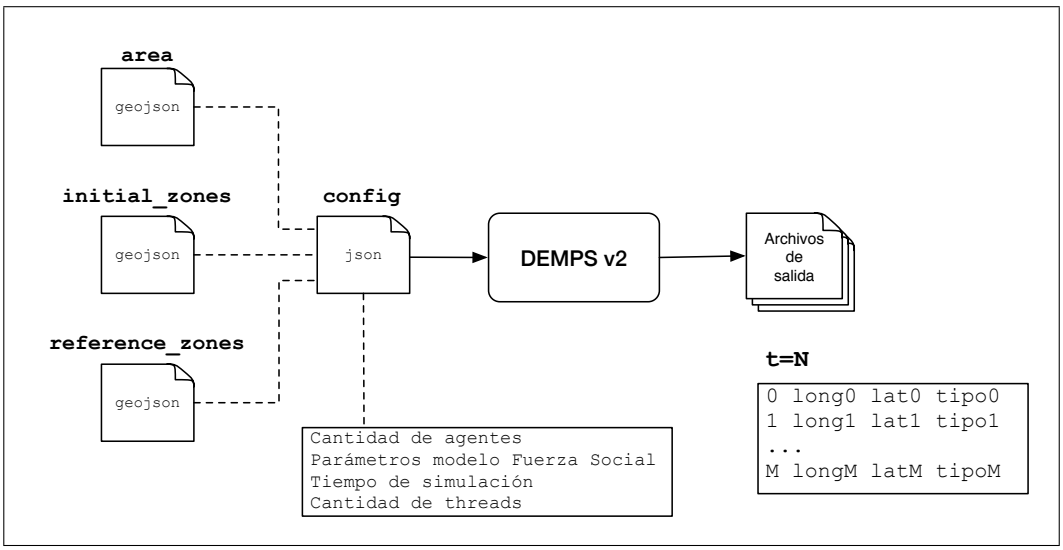

Figura 1: Diagrama global del software.

En la Figura 1 se muestra un diagrama global del software. La única entrada que tiene es un archivo de configuración JSON (`config.json`), que incluye las rutas al mapa donde se va a realizar la simulación (`area.geojson`), las zonas iniciales donde se van a localizar los agentes (`initial_zones.geojson`) y las zonas de encuentro o zonas seguras (`reference_zones.geojson`). Además, incluye la cantidad de agentes, parámetros del modelo de fuerza social, el tiempo de simulación, entre otros parámetros. Los resultados de la simulación, en términos de posición geográfica de los agentes, se almacenan en archivos, correspondientes a intervalos regulares de tiempo de simulación. Cada archivo, contiene la localización geográfica de cada agente en el instante de simulación respectivo. Por ejemplo, si el tiempo de simulación es de 1000 segundos, con 30000 agentes y los resultados de la simulación son cada 10 segundos, entonces se obtendrían $1000/10 = 100$ archivos, y cada uno de ellos tendría 30000 líneas, una por cada agente.

Los componentes internos más representativos del software se muestran en la Figura 2. El **Environment** es un objeto que permite administrar a todos los agentes de la simulación y

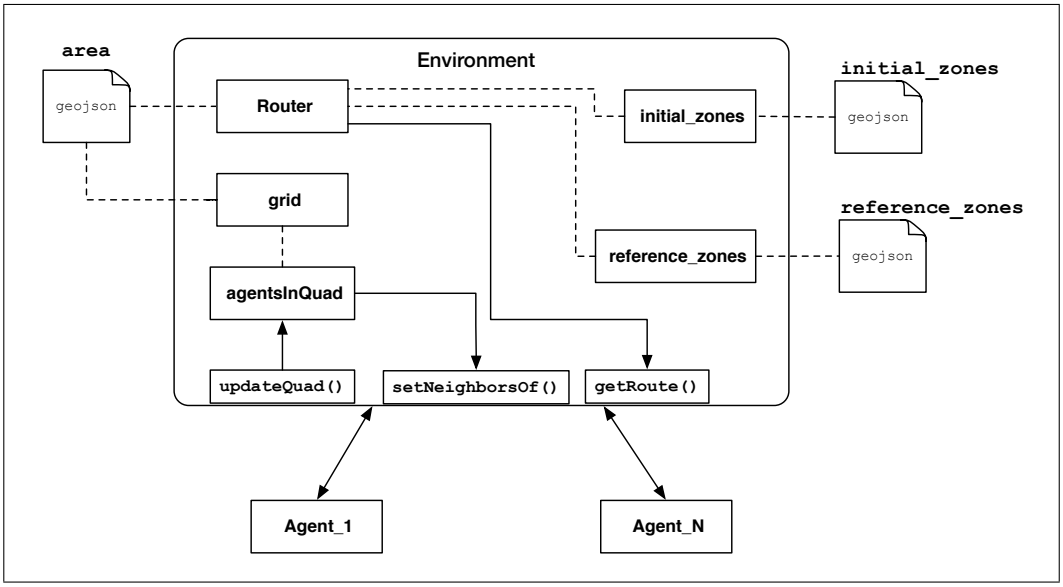

Figura 2: Componentes internos.

conectarlos adecuadamente con los mapas y zonas previamente definidos. El objeto **Router** determina que el movimiento de los agentes sea dentro de los caminos establecidos en el mapa de la simulación. A su vez, se crea una estructura, denominada **grid**, que divide el mapa en *cuadrantes*, cada uno con un identificador propio. Cada agente, a través de las coordenadas de su posición, determina en qué cuadrante se localiza y el ambiente guarda este dato en la estructura **agentsInQuad**. Con esto, es posible determinar los vecinos de un agente determinado, sin revisar la totalidad de agentes en el Environment.

3. Implementación

Los componentes del modelo (agentes, ambiente, router, zonas, etc), están implementadas en C++ bajo el estándar C++14. El diagrama de clases respectivo se muestra en la Figura 3. La clase **Simulator** es la que ejecuta la simulación. El resto de las clases implementa los componentes internos que se visualizan en la Figura 2. La librería OSRM[1] (Luxen & Vetter, 2011) permite calcular los desplazamientos de los agentes dentro del mapa, el que es obtenido desde la plataforma OpenStreetMap[2]. La librería GeographicLib[3] permite realizar conversiones entre coordenadas geográficas WGS84 (World Geodetic System 1984), utilizadas por los mapas, a coordenadas cartesianas ENU (East, North, UP), que es el sistema utilizado por el simulador. Finalmente, la librería CGAL[4] (Fabri & Teillaud, 2011), se utiliza para determinar posición de agentes dentro de las zonas que son representadas a través de polígonos, generar puntos dentro de las zonas de referencia e iniciales, etc.

La simulación se ejecuta en tres etapas: una fase de inicialización, una de calibración y el

[1]Open Source Routing Machine, http://project-osrm.org
[2]http://www.openstreetmap.org
[3]https://geographiclib.sourceforge.io
[4]Computational Geometry Algorithms Library, https://www.cgal.org

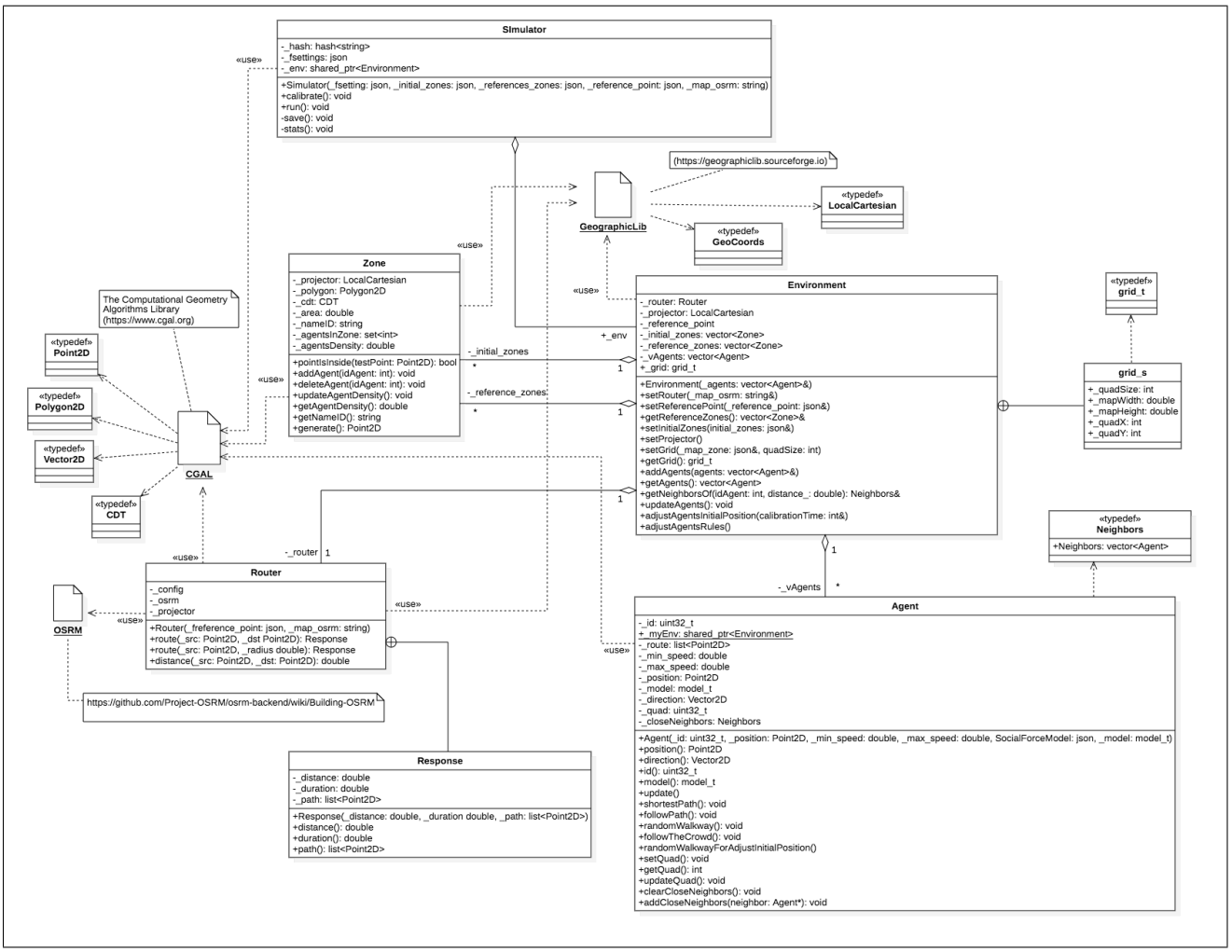

Figura 3: Diagrama de clases.

ciclo asociado al paso del tiempo de simulación. La fase de inicialización consiste en la lectura del archivo de configuración, de los mapas especificados en él y la creación de los objetos a utilizar en la simulación (Environment, Router, Zones y Agents). La fase de calibración mueve los agentes hacia las calles del mapa, debido a que inicialmente se localizan en forma aleatoria dentro de la zona inicial, no importando su su posición es en la calle o dentro de una casa, por ejemplo. El ciclo de avance de tiempo de simulación ejecuta el código para los pasos de simulación en forma repetitiva hasta un número especificado de segundos. El pseudo código para este ciclo se muestra en la Figura 4.

```
Por cada paso de simulación:
    Por cada agente:
       1) Actualizar su posición en base al Modelo de Fuerza Social y al mapa
       2) Actualizar información en el Environment sobre su cuadrante actual
```

Figura 4: Pseudo código para el ciclo de simulación.

4. Implementación con OpenMP

Para paralelizar el ciclo de simulación mostrado en la Figura 4 se utilizó OpenMP, que es un conjunto de directivas del compilador, librerías y variables de ambiente que permiten obtener paralelismo a alto nivel en programas escritos en Fortran y C/C++ (OpenMP, 2008). En particular, se utilizó la directiva *parallel for loop* para paralelizar los ciclos de los agentes. Esto permite que las iteraciones se realice a través de cierta cantidad de threads, que se ejecutan en distintos procesadores.

La segunda etapa del paso de simulación es complicada desde el punto de la programación con multi-thread, debido a que accesa en forma concurrente a estructuras en memoria compartida. Debido a esto, el SpeedUp no incrementa en forma lineal a medida que se aumentan los threads de ejecución.

Para visualizar el comportamiento del simulador con la paralelización con OpenMP, se ejecutó un escenario con un tiempo de simulación de $t_{sim} = 3000[s]$. El rango de agentes es $Ag = \{10000, 20000, \ldots, 90000\}$ e inicialmente la cantidad de threads es de $Th = \{1, 2, 4, 8\}$. Los resultados se muestran en la Figura 5a(1-3). Si bien se observa un aumento en el speedUp a medida que se aumentan los threads (Figura 5a.2), el gráfico de eficiencia (Figura 5a.3) muestra que el sistema no escala cuando se ejecuta con 8 o más threads (eficiencia menor o igual que 0.5)

5. Implementación con RepastHPC

Además del desarrollo del simulador en C++, se realizó una implementación en el entorno RepastHPC (Collier & North, 2013), el que ofrece clases abstractas y librerías para el desarrollo de simuladores basados en agentes, con soporte para modelos espaciales. El tipo de paralelización que ofrece Repast es en base a procesos. Está pensado para ambientes distribuidos con alguna implementación de MPI. Para el caso de este trabajo, se utilizó con procesos dentro de un mismo nodo. El tipo de descomposición del espacio de simulación se basa en "paralelización

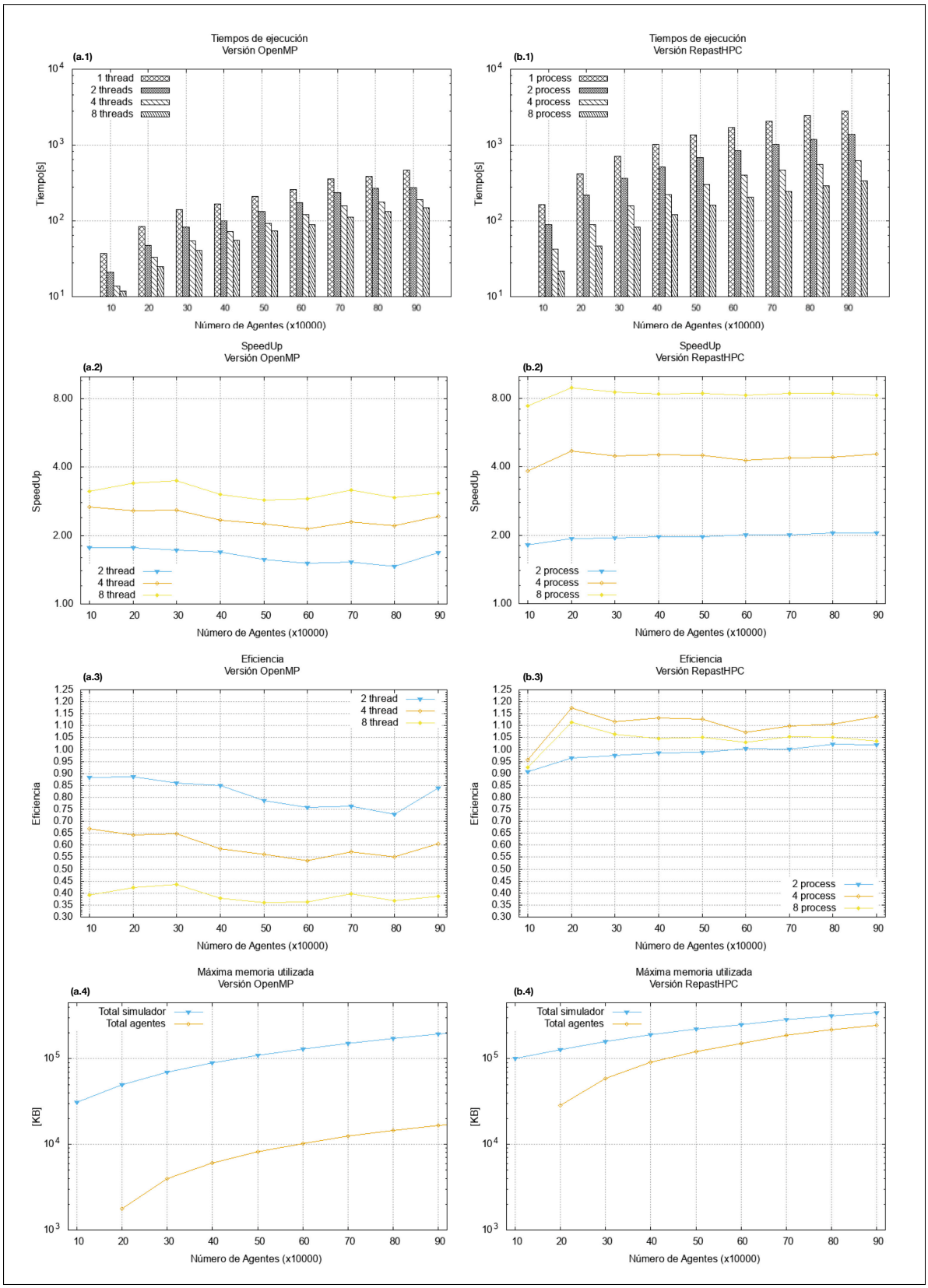

Figura 5: Desempeño del simulador con OpenMP y RepastHPC.

de agentes". Por ejemplo, si el escenario se divide en una grilla de 4×4 cuadrantes y la simulación se paraliza con 4 procesos, entonces cada proceso maneja 4 agentes, los que se mueven en la totalidad del escenario (Figura 6).

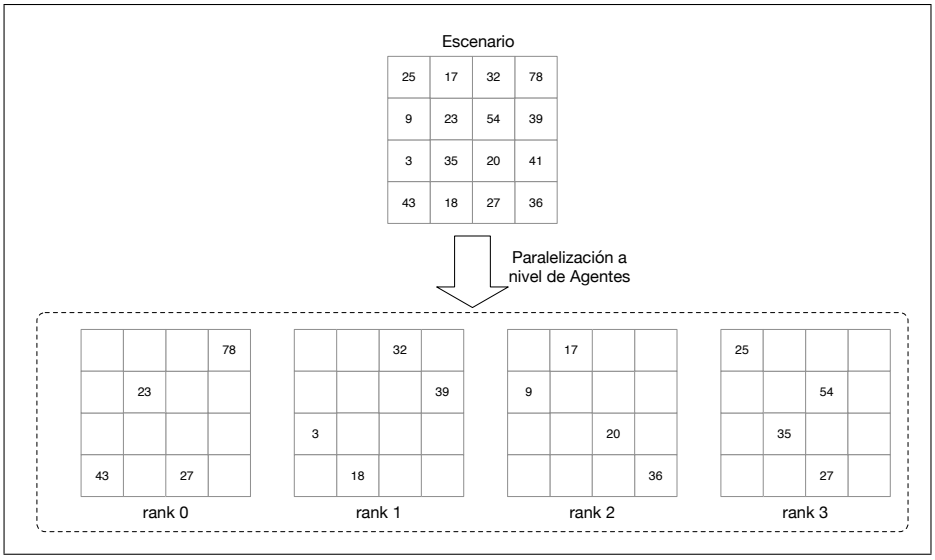

Figura 6: Paralelización a nivel de agentes.

Con el fin de obtener el comportamiento de esta versión del simulador, al igual que en la versión con OpenMP, se ejecutó un escenario con un tiempo de simulación de $t_{sim} = 3000[s]$. El rango de agentes es $Ag = \{10000, 20000, \ldots, 90000\}$ e inicialmente la cantidad de threads es de $Th = \{1, 2, 4, 8\}$. Los resultados se muestran en la Figura 5b(1-3).

Los resultados indican que esta versión es aproximadamente el doble más lenta que la versión con OpenMP, comparando ambos tiempos de simulación para el caso de 1 Proceso para la versión Repast y 1 Thread para la versión OpenMP. En la caso del SpeedUp, RepastHPC tiene un mejor desempeño, logrando valores superiores a OpenMP. Esto se debe a que las estructuras que posee este framework permiten un mejor escalamiento de las aplicaciones que se desarrollan sobre él. En el caso de la implementación realizada, la eficiencia alcanzada es ideal, por lo que indica que hay que revisar la comunicación entre los procesos, ya que se puede estar realizando en forma errónea.

5.1. Utilización de memoria

En la Figura 5a.4 y 5b.4 se muestra el costo espacial para el simulador, ejecutado en forma serial, con un rango de agentes de $Ag = \{10000, 20000, \ldots, 90000\}$, para ambas versiones. Como es de esperar, el gasto total en memoria, aumenta a medida que se simulan más agentes. La cantidad de memoria utilizada por los agentes es menor para el caso de la versión OpenMP, debido a que esta versión utiliza estructuras hechas a la medida para el problema, mientras que RepastHPC tiene conjunto de herramientas genéricas que deben ser instanciadas para implementar el simulador.

6. Conclusión

En este documento se ha presentado un simulador basado en agentes que permite evaluar tiempos de evacuación, tasa de ocupación de las zonas destinadas al encuentro de personas. Eventualmente, se puede utilizar para analizar flujos de personas en rutas de evacuación. A través de diversas mediciones, se logra concluir que el diseño propuesto puede escalar hasta 4 procesadores en forma eficiente. Esto se debe principalmente a que las estructuras que permiten administrar a los agentes son recursos compartidos entre los distintos threads de ejecución, y por lo tanto, son zonas críticas del código.

El simulador se implementó en dos versiones. Una utilizando C++ en forma pura, sin un framework, pero siguiendo la metodología de desarrollo propuesta por Macal & North (2015). La segunda, utilizando C++ con el framework RepastHPC (Collier & North, 2013). En los experimentos realizados, la versión pura obtuvo mejores tiempos de ejecución, mientras que la versión con RepastHPC obtuvo mejores índices en SpeedUp y Eficiencia.

La segunda etapa implementar los módulos para entregar datos de las tasas de ocupación de las zonas de encuentro, tiempos de evacuación, etc. Además, hay que determinar la estrategia para paralelizar bajo el paradigma de memoria distribuida. Para esto, se utilizará MPI y el framework Repast-HPC, que son utilizado por diversos autores para realizar simulaciones en ambientes distribuidos.

Referencias

Collier, N. & North, M. (2013). Parallel agent-based simulation with Repast for High Performance Computing. *SIMULATION*.

Fabri, A. & Teillaud, M. (2011). Cgal-the computational geometry algorithms library. In *10e colloque national en calcul des structures*, (pp.6̃).

Fachada, N., Lopes, V. V., Martins, R. C., & Rosa, A. C. (2017). Parallelization Strategies for Spatial Agent-Based Models. *International Journal of Parallel Programming*.

Helbing, D. & Molnar, P. (1998). Social Force Model for Pedestrian Dynamics. *Physical Review E*, *51*(5), 4282–4286.

Li, Z., Guan, X., Li, R., & Wu, H. (2016). 4D-SAS: A Distributed Dynamic-Data Driven Simulation and Analysis System for Massive Spatial Agent-Based Modeling. *ISPRS International Journal of Geo-Information*.

Luxen, D. & Vetter, C. (2011). Real-time routing with openstreetmap data. In *Proceedings of the 19th ACM SIGSPATIAL international conference on advances in geographic information systems*, (pp. 513–516). ACM.

Macal, C. & North, M. (2015). Introductory tutorial: Agent-based modeling and simulation. In *Proceedings - Winter Simulation Conference*, volume 2015-Janua, (pp. 6–20).

OpenMP, A. (2008). Openmp application program interface v3. 0. *OpenMP Architecture Review Board*.

Quezada, A. & Canesa, E. (2010). Modelado basado en agentes: Una herramienta para complementar el análisis de fenómenos sociales. *Avances en Psicologia Latinoamericana*, *28*(2), 226–238.

Thiele, J. C., Kurth, W., & Grimm, V. (2014). Facilitating parameter estimation and sensitivity analysis of agent-based models: A cookbook using NetLogo and R. *JASSS*.